CELL 6 Optimization Recommendations

> 📚 ***Related Documents***: - ***CELL6_FLOW_AND_OPTIMIZATION_GUIDE.md***: *Comprehensive explanation of how CELL 6 works, data flow, and technical details - This document: Quick-start guide with specific recommended settings*
>
> 💡 ***When to Use This Document***: *Use this guide when you want specific recommended settings for a common scenario. For a deeper understanding of how the system works, see the Flow Guide.*

Quick Reference: Most Efficient Settings

Scenario: Everything EXCEPT Ratings Given

Goal: Get basic metrics, grade metrics, last4 metrics, and pool metrics, but skip ratings given metrics.

Most Efficient Configuration:

```
# In pipeline_config.py:
CELL6_OPTIMIZE = {
    # Data Quality (skip if data already clean - saves 1-2 hours)
    'check_data_quality': False,      # ✅ Set False if data already cleaned
    'report_duplicates': False,        # ✅ Set False to skip reporting
    'report_overlaps': False,          # ✅ Set False to skip reporting
    'clean_duplicates': True,         # ✅ Keep True (fast operation)

    # Optional Metrics
    'include_ratings_given': False,   # ✅ Set False (you don't want this)
    'include_grade_metrics': True,     # ✅ Set True (you want this)
    'include_last4_metrics': True,    # ✅ Set True (you want this)

    # Pool Metrics
    'include_pool_metrics': True,     # ✅ Set True (you want this)
    'use_optimized_pool_metrics': True,  # ✅ Keep True (optimized path)

    # Performance Profiling
    'profile_performance': False,      # ✅ Set False (unless debugging)
}

# Cell execution flags (also in pipeline_config.py)
CELL6A = True   # ✅ Required - computes basic metrics
CELL6B = True   # ✅ Required - adds grade + last4 metrics
CELL6C = True   # ✅ Required - adds pool metrics
```

What This Does:

1. **CELL 6A** (~30 min):
    - Computes: Basic cumulative metrics (fresh computation)
    - Skips: Data quality checks (saves 1-2 hours)
    - Output: `df_pipeline_03a_basic_metrics.feather`
2. **CELL 6B** (~30-60 min):
    - Loads: `df_pipeline_03a_basic_metrics` (from 6A)
    - Recomputes: Basic metrics (redundant, ~30 min)
    - Adds: Grade metrics + Last4 metrics (NEW)

- Skips: Ratings given (not wanted)
- Skips: Data quality checks (already done in 6A)
- Output: `df_pipeline_03b_optional_metrics.feather`

3. **CELL 6C** (~1-2 hours):
   - Loads: `df_pipeline_03b_optional_metrics` (from 6B)
   - Recomputes: Basic metrics (redundant, ~30 min)
   - Skips: Grade + Last4 metrics (sets to False - avoids recomputing)
   - Skips: Ratings given (not wanted)
   - Adds: Pool metrics (NEW, ~1-1.5 hours)
   - Skips: Data quality checks (already done in 6A)
   - Output: `df_pipeline_03_oer_metrics.feather` (FINAL)

Total Runtime: ~2-3 hours - 6A: ~30 min (basic metrics) - 6B: ~30-60 min (recomputes basic + adds grade/last4) - 6C: ~1-2 hours (recomputes basic + adds pool)

Redundancy: ~60-90 minutes (basic metrics recomputed in 6B and 6C)

Time Saved: ~1-2 hours (by skipping data quality checks)

Why This Configuration is Most Efficient

1. Skips Data Quality Checks (Biggest Time Saver)

Setting `check_data_quality=False` saves **1-2 hours** in CELL 6A. This is the biggest optimization.

When to use: If your data is already clean (no duplicates, no overlaps), or if you've already run data quality checks.

2. Skips Ratings Given (Saves ~30 min)

Setting `include_ratings_given=False` means: - CELL 6B won't compute ratings given metrics (~30 min saved) - CELL 6C won't compute them either (already skipped)

3. Preserves Grade and Last4 Metrics

Even though CELL 6C sets `include_grade_metrics=False` and `include_last4_metrics=False`, these columns are **preserved** in the output because: - They exist in the input DataFrame (from 6B) - The function adds new columns but doesn't drop existing ones - So grade and last4 metrics from 6B are carried through to the final output

4. Minimizes Redundancy

The redundancy (recomputing basic metrics in 6B and 6C) is unavoidable because: - The function doesn't have an "add-only" mode - But we minimize it by skipping expensive operations (data quality checks)

Alternative: Skip CELL 6B Entirely?

Question: Can you skip 6B and get grade + last4 metrics in 6C?

Answer: **No, not efficiently**. Here's why:

If you skip 6B: - CELL 6C loads `df_pipeline_03a_basic_metrics` (from 6A) - CELL 6C detects basic metrics exist - CELL 6C sets `include_grade_metrics=False` and `include_last4_metrics=False` (hardcoded) - Result: Grade and last4 metrics are **not computed**

If you want grade and last4 metrics, you **must run CELL 6B**.

Performance Breakdown

With Recommended Settings

Cell	Operation	Time	Redundant?
6A	Basic metrics (fresh)	~30 min	No
6A	Data quality checks	**SKIPPED**	N/A
6B	Basic metrics (recompute)	~30 min	✅ Yes
6B	Grade + Last4 metrics	~30 min	No
6B	Ratings given	**SKIPPED**	N/A
6C	Basic metrics (recompute)	~30 min	✅ Yes
6C	Grade + Last4 metrics	**SKIPPED** (preserved from 6B)	N/A
6C	Pool metrics	~1-1.5 hours	No

Total: ~2-3 hours **Redundancy**: ~60 min (basic metrics recomputed twice) **Time Saved**: ~1-2 hours (data quality checks) + ~30 min (ratings given) = **~1.5-2.5 hours**

Comparison: With vs. Without Optimizations

Without Optimizations (Default)

```
CELL6_OPTIMIZE = {
    'check_data_quality': True,       # ❌ Runs expensive checks
    'include_ratings_given': True,     # ❌ Computes unwanted metrics
    'include_grade_metrics': True,
    'include_last4_metrics': True,
    'include_pool_metrics': True,
    'use_optimized_pool_metrics': True,
}
```

Runtime: ~4-5 hours - 6A: ~2-3 hours (includes data quality checks) - 6B: ~1 hour (includes ratings given) - 6C: ~1-2 hours

With Optimizations (Recommended)

```
CELL6_OPTIMIZE = {
    'check_data_quality': False,      # ✅ Skip expensive checks
    'include_ratings_given': False,   # ✅ Skip unwanted metrics
    'include_grade_metrics': True,
    'include_last4_metrics': True,
    'include_pool_metrics': True,
    'use_optimized_pool_metrics': True,
}
```

Runtime: ~2-3 hours - 6A: ~30 min (skips data quality checks) - 6B: ~30-60 min (skips ratings given) - 6C: ~1-2 hours

Time Saved: ~2 hours (40-50% faster)

Resumability: Running Cells Multiple Times

Key Point: Once a cell has been run once, you can run subsequent cells multiple times without re-running earlier cells.

How Resumability Works

Each cell checks for its own output file first. If found, it skips computation entirely. If not found, it loads the previous cell's output and recomputes.

6A → 6B Resumability

Once 6A has been run once: - ✅ You can run 6B **many times** without re-running 6A - ✅ If 6B output exists, 6B loads it and skips computation - ✅ If 6B output was deleted, 6B loads 6A output and recomputes - ✅ You can skip 6A entirely in future sessions (as long as `df_pipeline_03a_basic_metrics.feather` exists)

Example Workflow: 1. Run 6A once → creates `df_pipeline_03a_basic_metrics.feather` 2. Run 6B → loads 6A output, computes, saves `df_pipeline_03b_optional_metrics.feather` 3. Delete 6B output and run 6B again → loads 6A output again, recomputes 4. Run 6B again → finds 6B output, skips computation entirely

6B → 6C Resumability

Once 6B has been run once: - ✅ You can run 6C **many times** without re-running 6B - ✅ If 6C output exists, 6C loads it and skips computation - ✅ If 6C output was deleted, 6C loads 6B output and recomputes - ✅ You can skip 6B entirely in future sessions (as long as `df_pipeline_03b_optional_metrics.feather` exists)

Example Workflow: 1. Run 6B once → creates `df_pipeline_03b_optional_metrics.feather` 2. Run 6C → loads 6B output, computes, saves `df_pipeline_03_oer_metrics.feather` 3. Delete 6C output and run 6C again → loads 6B output again, recomputes 4. Run 6C again → finds 6C output, skips computation entirely

Summary

Scenario	Can Skip?	What Happens
Run 6B after 6A	✅ Yes (6A already run)	6B loads 6A output
Run 6B multiple times	✅ Yes	If 6B output exists, skips; if deleted, loads 6A output
Run 6C after 6B	✅ Yes (6B already run)	6C loads 6B output
Run 6C multiple times	✅ Yes	If 6C output exists, skips; if deleted, loads 6B output
Skip 6A, run 6B	❌ No	6B requires 6A output (will error)
Skip 6B, run 6C	✅ Yes (if 6A exists)	6C falls back to 6A output (but won't have optional metrics)

Bottom Line: The pipeline is designed for maximum resumability. Each cell can be run independently as long as its input (previous cell's output) exists.

Why Run 6A If 6B Recomputes Everything?

Great Question! You're right that 6B recomputes everything 6A does. Here's why 6A exists as a separate cell:

1. 6B Requires 6A Output (Current Design Limitation)

The code currently requires 6A output - 6B will **error** if `df_pipeline_03a_basic_metrics.feather` doesn't exist:

```
# CELL 6B code:
try:
    df_base = load_feather('df_pipeline_03a_basic_metrics')
except FileNotFoundError:
    print_v("❌ ERROR: df_pipeline_03a_basic_metrics.feather not found!")
    print_v("   Please run CELL 6A first to create this file.")
    raise  # ← Stops execution
```

So you can't skip 6A - it's a hard requirement in the current design.

2. Design Philosophy: Modularity Over Efficiency

The split was designed for: - **Resumability**: If 6B crashes, you still have 6A output - **Flexibility**: Run just 6A if you only need basic metrics (skip 6B/6C) - **Testing**: Test basic metrics independently - **Troubleshooting**: Isolate which part fails

The trade-off: **Modularity and resumability** are prioritized over **eliminating redundancy**.

3. Practical Benefits

Even though 6B recomputes basic metrics, having 6A separate provides: - **Checkpoint**: If 6B crashes after 1 hour, you don't lose the 6A work - **Selective execution**: Run 6A only if you just need basic metrics (~30 min) - **Debugging**: Test if basic metrics work before adding optional metrics

4. Could This Be Improved?

Yes! The design could be improved to allow 6B to run directly from CELL 5 output:

```
# Hypothetical improved 6B:
try:
    df_base = load_feather('df_pipeline_03a_basic_metrics')
except FileNotFoundError:
    # Fallback to CELL 5 output
    try:
        df_base = load_feather('df_pipeline_02_oer_joined')
        print_v("  • Loading from CELL 5 output (will compute basic + optional)")
    except FileNotFoundError:
        raise
```

This would allow skipping 6A if you're going to run 6B anyway, **saving ~30 minutes**.

However, this improvement hasn't been implemented yet, so **you must run 6A before 6B**.

Final Answer

Most Efficient Settings for Your Needs:

```
CELL6_OPTIMIZE = {
    'check_data_quality': False,      # Skip if data already clean
    'report_duplicates': False,
    'report_overlaps': False,
    'clean_duplicates': True,         # Keep True (fast)
    'include_ratings_given': False,   # Don't want this
    'include_grade_metrics': True,     # Want this
    'include_last4_metrics': True,    # Want this
    'include_pool_metrics': True,     # Want this
    'profile_performance': False,     # Set True only if debugging
}

CELL6A = True   # ⚠️ Required - 6B will error without it
CELL6B = True
CELL6C = True
```

Expected Runtime: ~2-3 hours (vs. ~4-5 hours without optimizations)

What You Get: - ✅ Basic cumulative metrics - ✅ Grade-based metrics - ✅ Last4 (rolling window) metrics - ✅ Pool metrics - ❌ Ratings given metrics (skipped)

Redundancy: ~60 minutes (unavoidable - basic metrics recomputed in 6B and 6C)

Time Saved: ~1.5-2.5 hours by skipping data quality checks and ratings given metrics

Note: You must run 6A even though 6B recomputes its work - this is a design limitation that prioritizes modularity and resumability over eliminating redundancy.

Resumability: Once 6A is run, you can run 6B multiple times without re-running 6A. Once 6B is run, you can run 6C multiple times without re-running 6B. See the "Resumability" section above for details.

Need More Details?

For a deeper understanding of: - How `CELL6_OPTIMIZE` controls each cell - Data flow diagrams - What gets recomputed vs. added - Multiple optimization strategies - Best practices for different scenarios

See **CELL6_FLOW_AND_OPTIMIZATION_GUIDE.md**.